

Java Raytracer

Report

Multimedia & Computer Graphics

Universidad Panamericana | 2026

Santiago Macias Fernandez

Introduction

This project is basically a raytracer built from scratch in Java. A raytracer is a rendering technique that simulates how light actually works in the real world, instead of rasterizing polygons like a regular game engine does, it shoots rays from the camera into the scene and calculates exactly what each ray hits and how it bounces around.

The result is that you get physically accurate shadows, reflections, and refractions kind of for free, because you're actually simulating the physics of light. The downside is that it's way slower than rasterization, but for images it gives much better results.

This report walks through how the project evolved from version 0.1 to version 1.0, what was added in each stage, and what the important technical decisions were. The codebase ended up being about 20 Java files and can render complex 3D scenes with textures, normal maps, reflections, and refractions.

v0.1 — Spheres

v0.1	Ray-Sphere Intersection
Files changed	Raytracer.java, Sphere.java, Vector3D.java, Ray.java, Camera.java, Scene.java, Object3D.java, Intersection.java

The very first version was just getting a single colored pixel on screen. Before anything else, the project needed a solid math foundation, which meant implementing Vector3D.java with all the operations that a raytracer needs: add, subtract, scalar multiply, dot product, cross product, magnitude, and normalize.

How ray casting works

The core idea is simple: for every pixel on screen, shoot a ray from the camera position through that pixel into the 3D scene. If the ray hits an object, color the pixel. If it doesn't, color it black (background).

The Camera class converts a pixel coordinate (x, y) into a normalized ray direction. It uses the field-of-view angle and the aspect ratio to map screen pixels to directions in 3D space. The formula is:

```
px = (2 * (x + 0.5) / width - 1) * aspectRatio * tan(fov/2)
py = (1 - 2 * (y + 0.5) / height) * tan(fov/2)
```

```
direction = normalize(px, py, -1)    // -1 because camera looks towards -Z
```

For spheres, the intersection math is a quadratic equation. The ray is parameterized as $P(t) = \text{origin} + t \cdot \text{direction}$, and you substitute that into the sphere equation ($\|P - \text{center}\|^2 = \text{radius}^2$) and solve for t . If t is positive, the sphere is in front of the camera. The smaller t is the closer intersection (front face of the sphere).



v0.2 — Triangles and Clipping

v0.2	Möller-Trumbore + Near/Far Clipping
Files changed	Triangle.java, Scene.java, Vector3D.java (cross product)

This version added the Triangle class, which is the building block for all 3D models. Almost everything in 3D graphics is made of triangles, any mesh, any OBJ file, is just a collection of triangles.

Möller-Trumbore algorithm

The algorithm to intersect a ray with a triangle is called Möller-Trumbore, and it's the standard approach in raytracers because it's efficient, it avoids computing the plane equation explicitly. The idea is:

- Compute $P = D \times \text{edge1}$ (cross product of ray direction and one edge)
- The determinant $= \text{edge2} \cdot P$ tells if the ray is parallel to the triangle

- Solve for barycentric coordinates u and v — if either is outside $[0,1]$ the hit point is outside the triangle
- Solve for t — the distance along the ray to the intersection

Barycentric coordinates are important: they give you the position inside the triangle as a weighted combination of the three vertices. This becomes crucial later for interpolating normals and UV coordinates.

Clipping planes

Clipping was also added here: a `nearClip` (0.1) and `farClip` (100.0) define a valid range for intersections. Any hit with $t < \text{nearClip}$ is behind or too close to the camera, and $t > \text{farClip}$ is too far. This avoids rendering objects behind the camera and also prevents floating-point precision issues.

v0.3 — OBJ File Reader

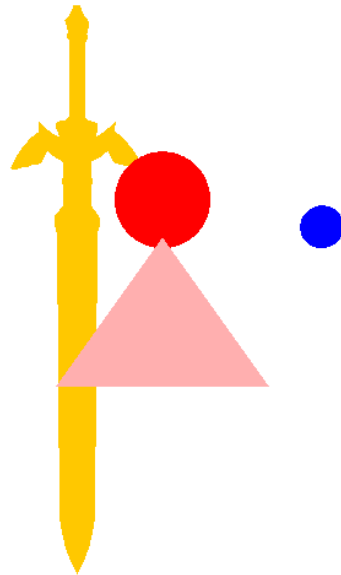
v0.3	Loading 3D Models from .obj Files
Files changed	OBJ.java

Having triangles is one thing, but you can't build a whole scene by hand-coding every vertex. The OBJ format is a simple text-based format that Blender and most 3D tools export. Each line starts with a keyword: `v` for vertex, `vn` for normal, `vt` for texture coordinate, `f` for face.

The OBJ reader parses each line and builds Triangle objects. Faces can be quads (4 vertices) or even n -gons, so the reader does fan triangulation, it takes the first vertex and connects it to every edge of the polygon, splitting it into triangles.

```
// Fan triangulation for quads/ngons
for (int i = 1; i < vIdx.length - 1; i++) {
    v0 = vertices[vIdx[0]];
    v1 = vertices[vIdx[i]];
    v2 = vertices[vIdx[i+1]];
    triangles.add(new Triangle(v0, v1, v2, color));
}
```

A **scale** parameter was also added so models that are exported at weird sizes can be **scaled** down when loading. Many OBJ files have coordinates in the hundreds or thousands, so a scale of 0.01 brings them to a reasonable size.



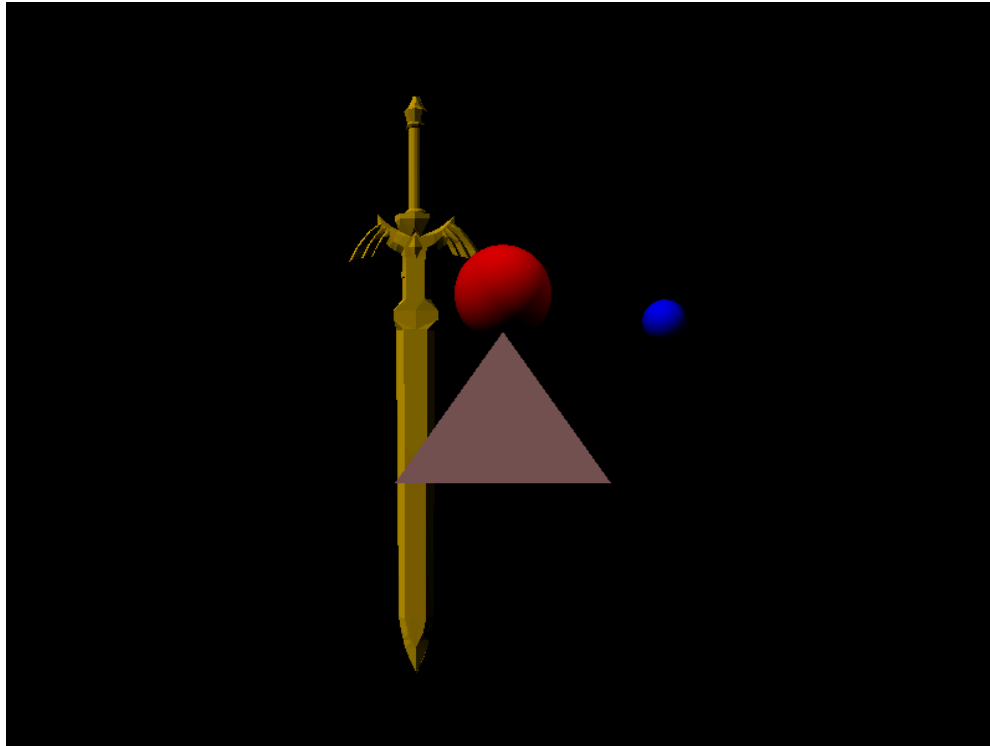
v0.4 — Flat Shading

v0.4	Lambert Diffuse Lighting
Files changed	Light.java, Raytracer.java (applyShading)

Up to this point everything was just solid flat colors and no lighting at all. Flat shading adds a basic diffuse light calculation using Lambert's law: the brightness of a surface depends on how directly it faces the light source.

The formula is: $\text{Diffuse} = \text{LC} \times \text{OC} \times \text{LI} \times \max(0, \text{N} \cdot \text{L})$, where LC is the light color, OC is the object color, LI is the light intensity, N is the surface normal, and L is the direction toward the light. The dot product $\text{N} \cdot \text{L}$ gives a value from -1 to 1 — negative values mean the face is pointing away from the light, so we clamp to 0.

A Light class was created with color and intensity. The direction of the light determines which faces are lit and which are in shadow.

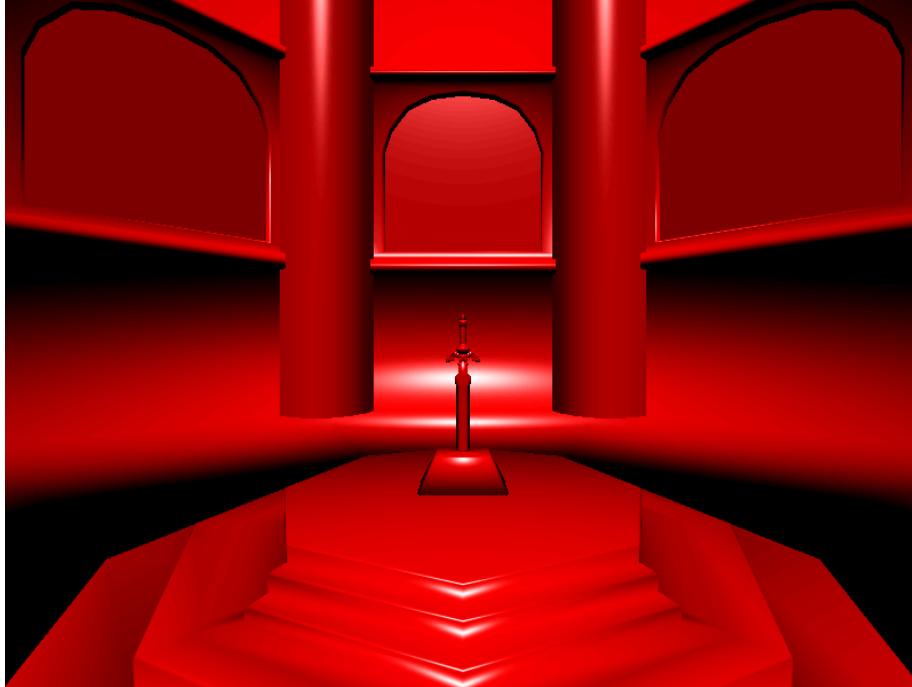


v0.5 — Phong Shading

v0.5	Per-vertex Normal Interpolation + Specular
Files changed	Triangle.java, OBJ.java, Raytracer.java

Flat shading gives a faceted look, each triangle is uniformly colored. Phong shading smooths this out by interpolating normals across the triangle surface. Instead of one normal per triangle, each vertex gets its own normal (read from `vn` in the OBJ file), and for any point inside the triangle the normal is interpolated using the barycentric coordinates from Möller-Trumbore.

This makes curved surfaces look smooth even though they're made of flat triangles. The specular highlight was also added using the reflection vector $R = 2 \cdot (N \cdot L) \cdot N - L$ and the view vector V , the specular term is $(R \cdot V)^{\text{shininess}}$. Higher shininess makes the highlight smaller and sharper.



v0.6 — Point Lights and Multiple Lights

v0.6	Light Hierarchy + Attenuation
Files changed	Light.java, DirectionalLight.java, PointLight.java, Scene.java

The original light was a simple directional light, same direction everywhere, no falloff. Point lights are more realistic: they have a position in 3D space and their intensity decreases with distance squared (inverse square law, same as real physics).

The Light class became abstract with two subclasses. DirectionalLight returns the same L direction for every point in the scene. PointLight calculates L dynamically as $\text{normalize}(\text{lightPos} - \text{intersectionPoint})$ and attenuates intensity as I / d^2 .

Scene.java was updated to hold a list of lights so multiple lights could be added. The shading loop iterates over all lights and accumulates their contributions.

v0.7 — Shadow Rays

v0.7	Hard Shadows via Occlusion Testing
-------------	---

Files changed`Scene.java (isOccluded), Raytracer.java`

To calculate shadows, after finding an intersection the raytracer shoots a second ray (the shadow ray) from the hit point toward each light source. If this ray hits any other object before reaching the light, that point is in shadow and the light is skipped.

Shadow acne

A classic problem here is shadow acne, the shadow ray can intersect the same surface it's coming from due to floating-point precision. The fix is to offset the shadow ray origin slightly along the normal by a small bias (0.001), so it doesn't immediately re-intersect its own surface.

For point lights, the shadow ray also needs a maximum distance, only objects between the surface and the light should cast shadows. Objects behind the light shouldn't matter.

v0.75 — Textures from MTL Files

v0.75**UV Mapping + Diffuse Texture Sampling****Files changed**`Material.java, MTL.java, Triangle.java, OBJ.java, Intersection.java, Raytracer.java`

This was a big architectural step. The OBJ format has a companion .mtl file that defines materials — including paths to texture images. The texture is a PNG or JPG that gets mapped onto the surface using UV coordinates.

UV coordinates (vt lines in the OBJ) are 2D coordinates in [0,1] range that map each vertex to a position on the texture image. Just like normals, they get interpolated with barycentric coordinates for every pixel inside a triangle. Then `sampleDiffuse(u, v)` does a simple pixel lookup on the image.

`Intersection.java` was updated to carry the interpolated `u` and `v` values. `Material.java` handles loading the image with `ImageIO` and sampling it. `MTL.java` parses the .mtl file and creates `Material` objects for each material name found.



v0.76 — BVH

v0.76	Bounding Volume Hierarchy
Files changed	AABB.java, BVHNode.java, Scene.java

Without any acceleration structure, every ray has to be tested against every single triangle in the scene. A scene with 100,000 triangles at 1080p resolution means about 2 trillion intersection tests per frame, and it is completely impractical.

The BVH (Bounding Volume Hierarchy) solves this by organizing objects into a tree. Each node has an AABB (axis-aligned bounding box) that contains all its children. If a ray misses the bounding box, the entire subtree is skipped.

Construction

The tree is built by: computing the AABB of all objects, finding the longest axis, sorting objects by their centroid on that axis, splitting at the median, and recursing on each half. Leaves contain at most 8 objects. This is $O(n \log n)$ and happens once before rendering.

AABB intersection (the slab method)

For each axis, compute when the ray enters (t1) and exits (t2) the pair of parallel planes. Track the latest entry and earliest exit across all three axes. If the latest entry is before the earliest exit, the ray hit the box. Pure arithmetic, no square roots.

v0.77 — Normal Maps (Bump Maps)

v0.77	TBN Matrix + Tangent-Space Normal Perturbation
Files changed	Material.java, MTL.java, Triangle.java, Raytracer.java

Normal maps allow a low-polygon mesh to look like it has much more geometric detail — things like brick grout, surface scratches, or skin pores — without actually adding geometry. The trick is to store per-pixel normal directions in a texture and use them to perturb the shading normal.

The TBN matrix

The normal map stores vectors in tangent space — a coordinate system local to each triangle where Z is the surface normal, X is the tangent (along U in UV space), and Y is the bitangent (along V in UV space). To use these in world space, a 3x3 TBN matrix is built from the tangent (T), bitangent (B), and normal (N) vectors.

T and B are calculated from the UV differences and edge vectors of the triangle. This math is done once per triangle in the constructor:

```
du1 = uv1[0]-uv0[0];  dv1 = uv1[1]-uv0[1];
du2 = uv2[0]-uv0[0];  dv2 = uv2[1]-uv0[1];
f = 1.0 / (du1*dv2 - du2*dv1);
tangent   = normalize(f * (dv2*edge1 - dv1*edge2));
bitangent = normalize(f * (-du2*edge1 + du1*edge2));
```

At shading time, the RGB values from the normal map are decoded from [0,255] to [-1,1], then transformed to world space: $N_world = tsN.x*T + tsN.y*B + tsN.z*N$. Before this, T is re-orthogonalized against the Phong-interpolated normal using Gram-Schmidt to keep everything consistent.



v0.8 — Blinn-Phong

v0.8	Hybrid Material Architecture
Files changed	BlinnPhongMaterial.java, TypicalMaterials.java, Triangle.java, Scene.java, Raytracer.java

Up to this point, light behavior (how shiny, how diffuse) was hardcoded in the shader. This version introduced a proper material system that separates two distinct concerns:

- `Material.java` — only holds textures from the MTL file (diffuse texture, normal map)
- `BlinnPhongMaterial.java` — only holds light behavior coefficients (k_a , k_d , k_s , shininess, transparency, reflectivity, i_o_r)

A `Triangle` can have both: its MTL `Material` provides the color and normal map, and a `BlinnPhongMaterial` is assigned when adding the mesh to the scene to control how it responds to light. This means you can load the same OBJ with different material behaviors without modifying the geometry.

Blinn vs Phong specular

The specular calculation was upgraded from classic Phong to Blinn-Phong. Instead of computing the reflect vector R and doing $\text{dot}(R, V)$, it computes a half-vector $H = \text{normalize}(L + V)$ and does $\text{dot}(N, H)$. This is slightly cheaper and avoids artifacts at grazing angles. The

TypicalMaterials class provides preset materials like GOLD, SILVER, IRON, GLASS, MARBLE, DIAMOND with realistic coefficient values.



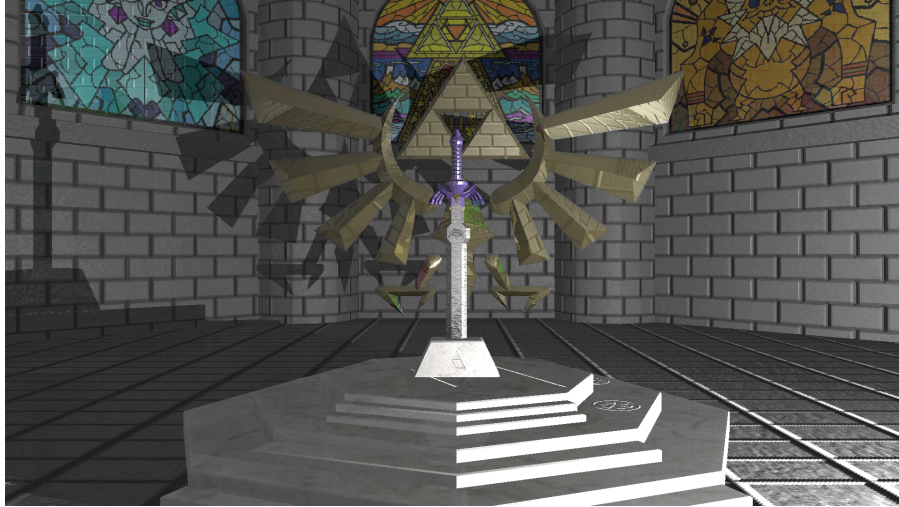
v0.9 — Recursive Reflections

v0.9	Mirror Reflections with Bounce Limit
Files changed	Raytracer.java

Reflections are what make raytracers look so much better than rasterizers. When a ray hits a reflective surface, instead of stopping, it spawns a new ray in the reflected direction and traces that recursively.

The reflected direction is: $R = I - 2 \cdot (I \cdot N) \cdot N$, where I is the incident ray direction. A MAX_BOUNCES constant (default 2-3) limits the recursion depth to avoid infinite bouncing between two mirrors.

The final color blends the local shading with the reflected color: $finalColor = (1 - kr) \cdot localColor + kr \cdot reflectedColor$. With $kr=1.0$ and no diffuse, you get a perfect mirror.



v1.0 — Refraction with Snell's Law and Fresnel

v1.0	Transparent Materials + IOR + Fresnel
Files changed	BlinnPhongMaterial.java, TypicalMaterials.java, Raytracer.java

Refraction is what makes glass, water, and gems look like glass, water, and gems. When a ray enters a transparent material, it bends according to Snell's Law: $n_1 \sin(\theta_1) = n_2 \sin(\theta_2)$, where n_1 and n_2 are the refractive indices (IOR) of the two materials.

IOR — Index of Refraction

The IOR was added as a seventh parameter to BlinnPhongMaterial. Real-world values were used in TypicalMaterials: GLASS=1.50, WATER=1.33, DIAMOND=2.42, EMERALD=1.57. Higher IOR means more bending and more distortion.

Snell's Law in vector form

```
// cosI = -dot(N, I)    (angle of incidence)
// ratio = n1/n2
// sinT2 = ratio^2 * (1 - cosI^2)
// if sinT2 > 1 → Total Internal Reflection (no refracted ray)
cosT = sqrt(1 - sinT2);
refractDir = ratio*I + (ratio*cosI - cosT)*N;
```

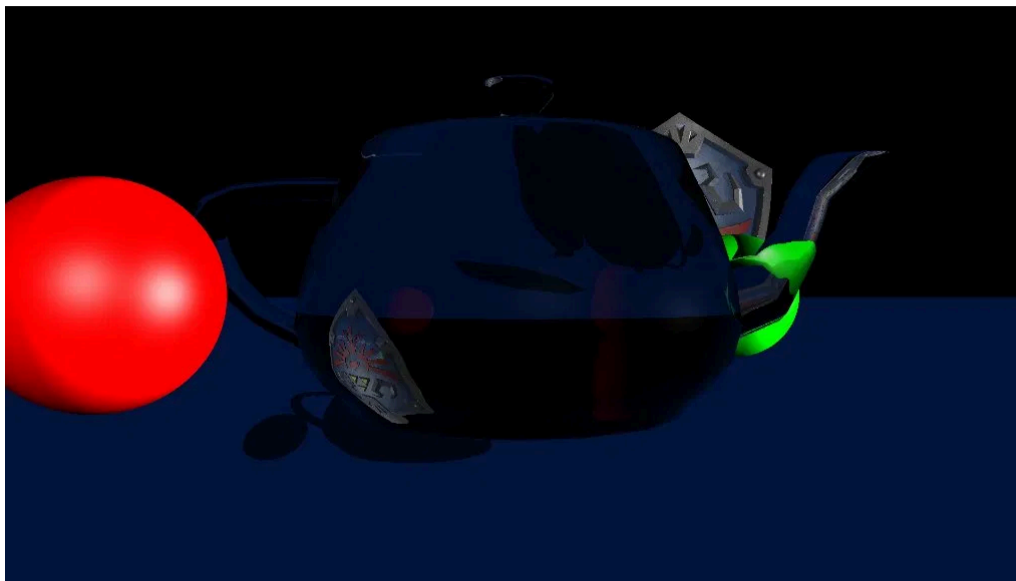
Fresnel effect (Schlick approximation)

Even transparent materials reflect more at grazing angles — this is the Fresnel effect, and it's why glass looks transparent from straight on but mirror-like from the side. The Schlick approximation computes this efficiently:

```
r0 = ((n1-n2)/(n1+n2))^2;
fresnelKr = r0 + (1 - r0) * (1 - cosI)^5;
```

The final color blend is: $\text{finalColor} = \text{localColor} * (1 - kr - kt) + \text{reflectedColor} * kr + \text{refractedColor} * kt$. The three contributions are energy-conserving — they sum to at most 1.

Total Internal Reflection is also handled: when a ray inside a dense material hits the boundary at too steep an angle, $\sin^2(T) > 1$ and there is no refracted ray — all light reflects back inside. This is what causes the characteristic sparkling in diamonds and gems.



Final Architecture

After all iterations, the codebase is organized as follows:

Core math

- Vector3D.java — all vector math (add, sub, dot, cross, normalize, magnitude)
- Ray.java — origin + direction, with pre-computed inverse direction for AABB

Scene and acceleration

- Scene.java — holds objects, lights, camera, BVH. Main raycast and shadow test entry points
- AABB.java — axis-aligned bounding box with slab-method ray intersection
- BVHNode.java — binary tree of AABBs built from scene objects

Geometry

- Object3D.java — abstract base class
- Triangle.java — Möller-Trumbore, Phong normals, UV interpolation, TBN, BlinnPhongMaterial
- Sphere.java — analytic ray-sphere intersection
- Plane.java — infinite plane for floors and walls

Materials and textures

- Material.java — diffuse texture + normal map loading and sampling
- MTL.java — .mtl file parser (map_Kd and map_Bump)
- BlinnPhongMaterial.java — ka, kd, ks, shininess, transparency, reflectivity, IOR
- TypicalMaterials.java — presets: GOLD, SILVER, GLASS, DIAMOND, STONE, etc.

Lighting

- Light.java — abstract base
- DirectionalLight.java — constant direction, no attenuation
- PointLight.java — position-based, intensity / d^2 attenuation

Rendering

- Camera.java — ray generation from pixel coordinates
- Raytracer.java — main render loop, traceRay() recursive function, applyShading() with full Blinn-Phong + normal maps + reflection + refraction + Fresnel

FINAL RENDERS AND EXPLANATION

RENDER 1 – The Temple and The Legend

With the first render, I wanted to do something I like, and what this render aims to tell is the phrase "all games have a story, but only one is a legend," and that's the story: the legend turned out to be real, and the sword rests on its pedestal, waiting to be discovered by the hero.



RENDER 2 – A Good Beginning...

We've all started something for the first time. I suppose almost everyone here knows Minecraft, and this render tries to convey the feeling of starting something for the first time and having everything go well (Steve just started and already has diamonds).

Which doesn't mean it will be perfect later on (represented by Herobrine in the mirror's reflection).



RENDER 3 – Knight's Dream

A knight fell defeated in his greatest battle, but all is not lost. The knight entered a dream where an angel appeared to him. The knight asked for help and strength to continue. This is the scene depicted.

The aim is to tell, through more fantastical objects and more saturated colors, the effect of a dream—a dream that will bring something good.



EXPLANATION OF GREATEST CHALLENGE

My biggest challenge was the phong shading, it looked weird and also like flat shading.

The problem was in the Möller–Trumbore algorithm, more specifically in the barycentric coordinates, I had inverted the V and U, so it scrambles the normals between vertices, producing incorrect interpolation that visually resembles flat shading or gives strange results.

The fix was given by a friend, that has the same problem, and say me that to fix that, i only need to invert the U and V in the formula

SELF EVALUATION

Ray Tracer v1.0 (10%)

- Shadows (5%)
- Blinn-Phong (specular & shininess) (5%)

In this part I would give me the 10% because my shadows and Blinn-Phong are working correctly.

Reflection Works (25%)

- Reflected objects are shown correctly (15%)
- It shows at least 2 bounces (10%)

I also would give me the 25%, also the reflections work as expected

Refraction Works (25%)

- Refracted objects are shown correctly (15%)
- Light changes direction correctly (10%)

I also would give me the 25%, also the Refraction work

Coding Principles (15%)

- Does it have unnecessary code? (1%)
- Does it properly use OOP? (2%)
- Is the code reusable? (1%)
- Is the code flexible? (1%)
- Does it have bugs? (2%)
- Is the code scalable? (1%)
- Does it have comments? (1%)
- Is the code a huge mess or neat? (1%)

Here i would give me a 14% or 13%, because i think that my code is a little bit a mess, especially in the part of the creations of the scenes. i have a lot of code (i think that is organized, but also i think it can be done in a cleaner way)

LaTeX Report (5%)

A full report in LaTeX explaining how you created the ray tracer and showing at least 5 images of how your ray tracer evolved. Include the explanation for each of the final images.

Evaluation:

- Description of the process of the ray tracer (1.5%)
- Explanation of each of the final images (1.5%)
- Explanation of greatest challenges (1%)

- Self-evaluation of all points (1%)

In this part i would give me a 5%, because i do all the points, including the process of the ray tracer, i explain each of the final images, i explain my greatest challenges and i auto-evaluate my self

3 Complex Final Renders (10%)

- Each render has a story (3%)
- Each render measures 4096 × 2160 and is PNG (2%)
- It contains something more than spheres or teapots (2%)
- It is aesthetic (3%)

here also i give me the 10%. my renders tell a story and also are 4K, they also have more than spheres and teapots and in my opinion its aesthetic.

CHALLENGE MODE

I think I can get 1.5% from the challenge mode of “Implement Rotation and Scale of Objects”, because I implement the scale of the objects.

I also would give me 5% of the extra points, because i also implement the Bounding Volume Hierarchy

And i also implement the UV reader + texture and normals maps, so i would give me that 8% extra